

Agent Platform Foundation - System Overview

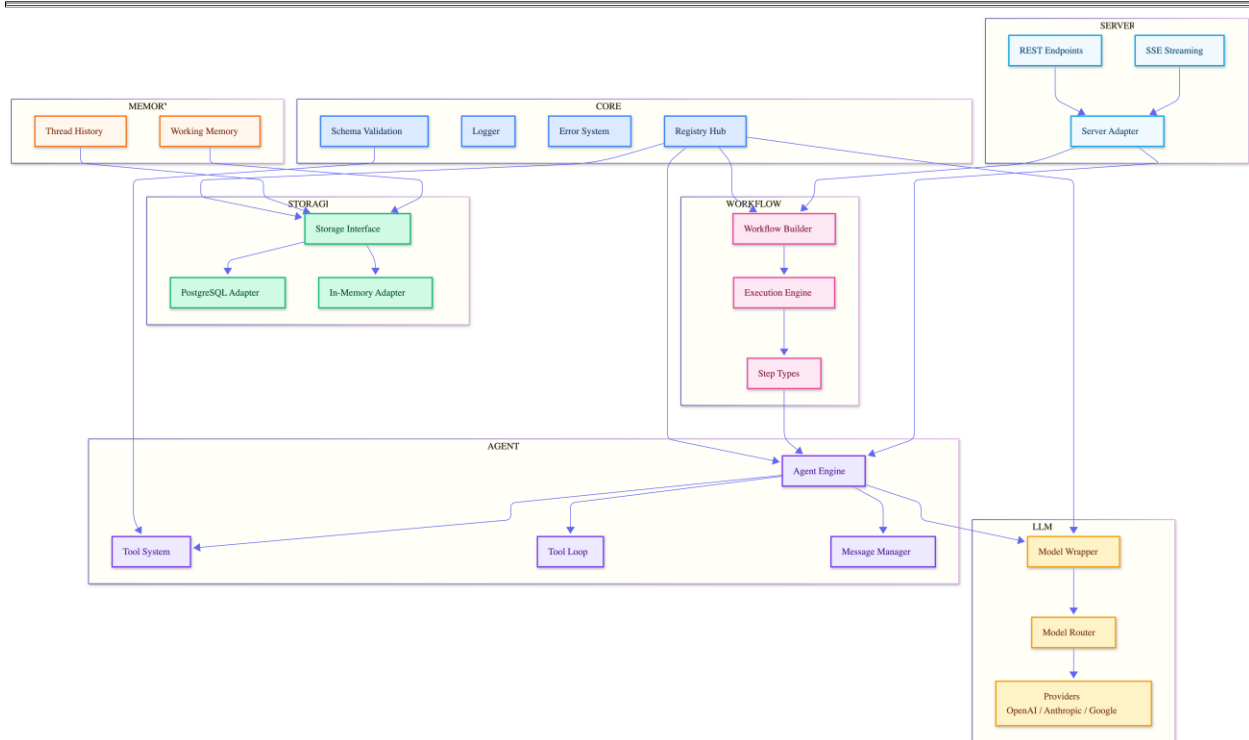
1. Overview

The Foundation Platform is the core infrastructure that enables teams to build AI agents and agent workflows. It provides a set of code-first SDK packages for teams to integrate by importing the SDK, defining agents/tools/workflows in Python, and registering them with the platform to run.

Design Principles

- **Code-first:** All components are defined in code, type-safe, testable, and version-controllable.
- **Pluggable:** Storage, LLM providers, and server frameworks are interchangeable via the adapter pattern.
- **Minimal & Composable:** Only essential building blocks are provided. Teams compose them to meet their needs.

Architecture: 7 Components



2. Component Details

2.1 Core Contracts

The foundation of the entire system. Teams do not interact with this directly; it runs under the hood to wire all components together.

Module	Description
Registry Hub	Registers and manages the lifecycle of all components (agents, tools, workflows, storage). Automatically inject dependencies when a component is registered.
Logger	Provides unified, structured logging across the system. Pluggable swap the logging backend without changing the application code.
Error System	Typed error hierarchy for consistent error handling and fast debugging across all modules.
Schema Validation	Validates tool and workflow step input/output using Zod schemas. Automatically converts schemas to JSON Schema for LLM tool calling.

2.2 LLM Abstraction

Abstraction layer for calling AI models without coupling to a specific provider.

Module	Description
Model Wrapper	Unified interface for all LLM providers. Exposes <code>generate ()</code> and <code>stream ()</code> regardless of whether the underlying provider is OpenAI, Anthropic, or Google.
Model Router	Routes requests to the correct provider. Supports switching between models and fallback when a provider fails.
Providers	Adapters for each specific LLM provider (OpenAI, Anthropic, Google). Handles authentication, rate limiting, and provider-specific request/response formatting.

2.3 Agent Runtime

The central component where AI agents are created and executed.

Module	Description
Agent Engine	Runs an agent with a system prompt, calls the LLM, and returns results via <code>generate()</code> (synchronous) or <code>stream()</code> (real-time). Supports structured output via schema.
Tool System	Allows defining tools using <code>createTool()</code> with input/output schemas and an <code>execute</code> function. The agent autonomously decides when to call a tool based on context.
Tool Loop	Automatically loops <code>execute tool</code> → <code>receive result</code> → <code>call LLM</code> again until the agent produces a complete answer. Enforce max iterations to prevent infinite loops.
Message Manager	Manages conversation threading, multi-turn chat, and message classification by role (user, assistant, system, tool).

2.4 Storage

Data persistence layer with a pluggable adapter architecture.

Module	Description
Storage Interface	Defines contracts for data domains (memory, workflow state, agent config). All adapters must implement this interface.
PostgreSQL Adapter	Production adapter. Persist data to PostgreSQL.
In-Memory Adapter	Development and testing adapters. Data is stored in RAM and lost on restart.

2.5 Workflow Engine

Enables composing multiple processing steps into a logical pipeline.

Module	Description
Workflow Builder	<code>createWorkflow()</code> API for defining workflows. Declares steps, execution order, and input/output mapping between steps.

Module	Description
Execution Engine	Runs workflows in the defined order. Supports sequential execution, conditional branching, and per-step error handling.
Step Types	Available step types: Function Step (custom logic), Agent Step (calls an agent), Workflow-in-Workflow (nested workflow composition).

2.6 Memory

Agent memory that maintains context across conversation turns.

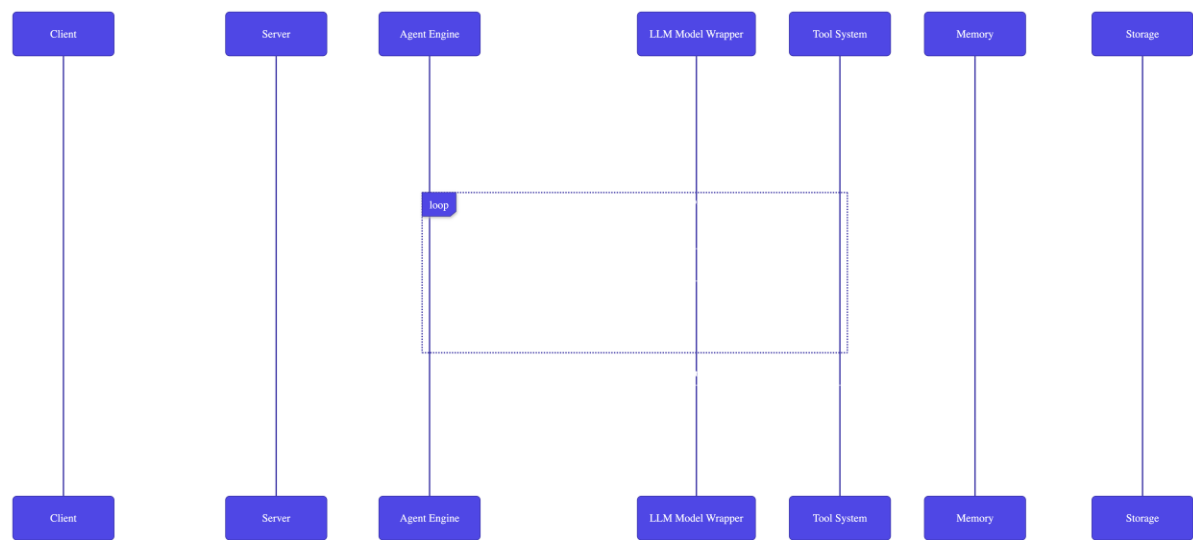
Module	Description
Thread History	Stores conversation history per thread. Agents retrieve previous messages to respond with context.
Working Memory	Short-term session memory. Stores temporary state (user preferences, extracted facts) that the agent needs to remember within a session.

2.7 Server

Exposes agents and workflows as HTTP APIs for end-users or external systems.

Module	Description
Server Adapter	Framework-agnostic abstraction layer. Plugs into Hono, Express, or other frameworks.
REST Endpoints	Automatically generates REST APIs for each registered agent and workflow (POST /agents/:id/generate, POST /workflows/:id/run).
SSE Streaming	Server-Sent Events support for stream(). Clients receive real-time token-by-token responses instead of waiting for the full result.

3. End-to-End Request Flow



4. Technology Stack

Layer	Technology	Purpose
Language	Python 3.12+	Primary development language
Web Framework	FastAPI + Uvicorn	Async HTTP server, auto-generated OpenAPI docs, SSE streaming
Validation	Pydantic v2	Input/output schema validation, JSON Schema generation for LLM tool calling
LLM Integration	LiteLLM	Unified multi-provider wrapper (OpenAI, Anthropic, Google, Azure, etc.)
Database ORM	SQLAlchemy 2.0 + asyncpg	Async PostgreSQL access, migration support
Migration	Alembic	Database schema versioning and migration
Testing	pytest + pytest-asyncio	Unit and integration testing with async support
Package Manager	uv	Fast Python package management and virtual environments
Linting	Ruff	Fast linter and formatter (replaces flake8, black, isort)
Type Checking	mypy / pyright	Static type analysis
Containerization	Docker	Consistent dev/prod environments

Layer	Technology	Purpose
CI/CD	GitHub Actions	Automated build, test, lint pipeline